

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Ion Moroșanu FAF-242

Report

Laboratory Work no. 1
Network Programming

Checked by:

_____
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
Hardware and software 3
1 THE PROGRAM **4**
1.1 The data 4
1.2 The structure of the program 4
1.3 Building and running 5
2 TASK 1A: COUNTING WITH THREADS **6**
2.1 Dividing the file 6
2.2 Results 6
2.3 Answers to the questions 7
3 TASK 1B: THREE WAYS TO COUNT **9**
3.1 Implementation 9
3.2 Results 9
3.3 Answers to the questions 10
4 TASK 1C: PRODUCER AND CONSUMERS **12**
4.1 Implementation 12
4.2 Results 12
4.3 Answers to the questions 14
5 TASK 2: MANY FILES ON SLOW STORAGE **16**
5.1 How the task was run 16
5.2 Results 16
5.3 Answers to the questions 17
6 ANSWERS TO THE CONTROL QUESTIONS **19**
CONCLUSIONS **21**
REFERENCES **22**
APPENDIX A. SOURCE CODE **23**

INTRODUCTION

Purpose of the work

The purpose of the work is to measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload [1]. The workload is the same in every task: counting the negative, zero and positive numbers among 50 000 000 integers stored in a text file.

The following tasks were set:

a) to count the numbers with 1 to 64 threads, each thread reading its own part of the file (Task 1a);

b) to count an array of numbers in three ways: shared counters without synchronisation, shared counters protected by a lock, and a separate counter for each thread (Task 1b);

c) to share the work between one reader thread and several worker threads through a bounded queue (Task 1c);

d) to read the same numbers from a USB flash drive and decide from the measurements whether the work is CPU-bound or I/O-bound (Task 2).

Hardware and software

All the measurements were made on the laptop described in Table 1.

Table 1 – The hardware and software used

Component	Value
Computer	Microsoft Surface Laptop 4
Processor	Intel Core i5-1145G7, 2.6 GHz base, up to 4.4 GHz
Cores	4 physical cores, 8 logical processors (Hyper-Threading)
Memory	16 GB LPDDR4x, 4267 MT/s
Disk	512 GB NVMe SSD (WD SDBPTPZ-512G)
USB drive	AI Mass Storage, 4 GB, FAT32, connected as USB 2.0 (High-Speed)
Operating system	Windows 11 Pro, build 26200
Language and runtime	C# 14, .NET 10.0.12 (SDK 10.0.401), Release build
Power	on battery, Windows Balanced power mode, no other heavy programs running

1 THE PROGRAM

1.1 The data

The input file numbers.txt is generated by the program itself with the command `gen`. It contains 50 000 000 integers from −1 000 000 to 1 000 000, one per line, produced by the `Random` class with the fixed seed 42, so the same file can be generated again. The file has 369 453 137 bytes (352 MB). For Task 2 the command `split` copies it, in order, into 8 files of 6 250 000 numbers each, about 44 MB per file.

The correct totals are 25 005 087 negative, 24 zero and 24 994 889 positive numbers, 50 000 000 in all. Every run of Tasks 1a, 1c and 2 and of versions 2 and 3 of Task 1b printed exactly these numbers.

1.2 The structure of the program

The whole lab is one C# file, Program.cs (Appendix A), that uses only the standard library of .NET. The first command-line argument selects the task, and the next ones give the input and the number of threads (Table 1.1).

Table 1.1 – The commands of the program

Command	What it does
`gen <file>`	generates the 50 000 000 numbers
`split <file> <dir>`	splits the file into 8 files
`1a <file> <threads>`	Task 1a
`1b <file>`	Task 1b, all three versions
`1c <file> <workers>`	Task 1c
`2 <dir> <threads>`	Task 2

Every thread is an operating system thread created with `new Thread(...)` and waited for with `Join()`. The method `Run` measures the work with a `Stopwatch` and then prints the totals and the peak memory of the process, read from `Process.PeakWorkingSet64` [4]. The time does not include the start of the process. Every configuration is a separate run of the program, so the peak memory of one configuration does not contain the memory of another one.

Tasks 1a and 2 parse raw bytes with the class `SignCounter`. It reads the bytes one by one, builds the value from the digits, and at the end of every line adds one to the negative, zero or positive counter. Its state is kept between calls, so a number may be split between two buffers. Task 1c receives lines of text, so its workers use `int.Parse`.

1.3 Building and running

The program needs the .NET 10 SDK. It is built and run with the commands below; `gen` and `split` are needed only once.

```
dotnet build -c Release
dotnet run -c Release -- gen numbers.txt
dotnet run -c Release -- split numbers.txt parts
dotnet run -c Release -- 1a numbers.txt 8
dotnet run -c Release -- 1b numbers.txt
dotnet run -c Release -- 1c numbers.txt 8
dotnet run -c Release -- 2 E:\prlab1 4
```

The script `run_task1.ps1` runs every configuration of Task 1 five times and writes the output to results\task1.txt. The script `run_task2.ps1 -Drive E` copies the 8 files to the USB drive and, before every run, waits until the drive is unplugged and plugged back in; its output goes to results\task2.txt. Every time in this report is the median of the runs of that configuration.

2 TASK 1A: COUNTING WITH THREADS

2.1 Dividing the file

The file is divided by bytes, not by lines, because the place where a line starts can be known only by reading everything before it. With n threads the cuts are made at size·i/n for i = 1, …, n − 1. Such a cut usually falls in the middle of a line, so every cut is moved forward to the start of the next line by the function `LineStart`, and a line belongs to the part in which it starts (see control question 2). Each thread computes its own two ends, opens its own `FileStream`, moves to its start and reads only its own bytes, in blocks of 64 KB, so no thread reads the whole file.

2.2 Results

Every configuration was run five times. In Task 1 the file had been read shortly before, so Windows kept it in its file cache (the file is 352 MB and the laptop has 16 GB of memory), and the reads came from memory, not from the SSD. Table 2.1 gives the median time, the speed-up relative to one thread and the median peak memory, and Figure 2.1 shows the same values.

Table 2.1 – Task 1a, median of five runs

Threads	Time, s	Speed-up	Peak memory, MB	Times of the five runs, s	Totals correct
1	1.38	1.00	25	1.72, 1.37, 1.32, 1.51, 1.38	yes
2	0.75	1.84	25	0.82, 0.74, 0.77, 0.75, 0.72	yes
4	0.44	3.12	25	0.67, 0.46, 0.41, 0.44, 0.40	yes
8	0.38	3.67	25	0.57, 0.64, 0.34, 0.35, 0.38	yes
16	0.34	4.06	26	0.51, 0.49, 0.32, 0.34, 0.31	yes
32	0.33	4.22	28	0.33, 0.50, 0.30, 0.28, 0.33	yes
64	0.33	4.12	29	0.45, 0.68, 0.33, 0.28, 0.28	yes

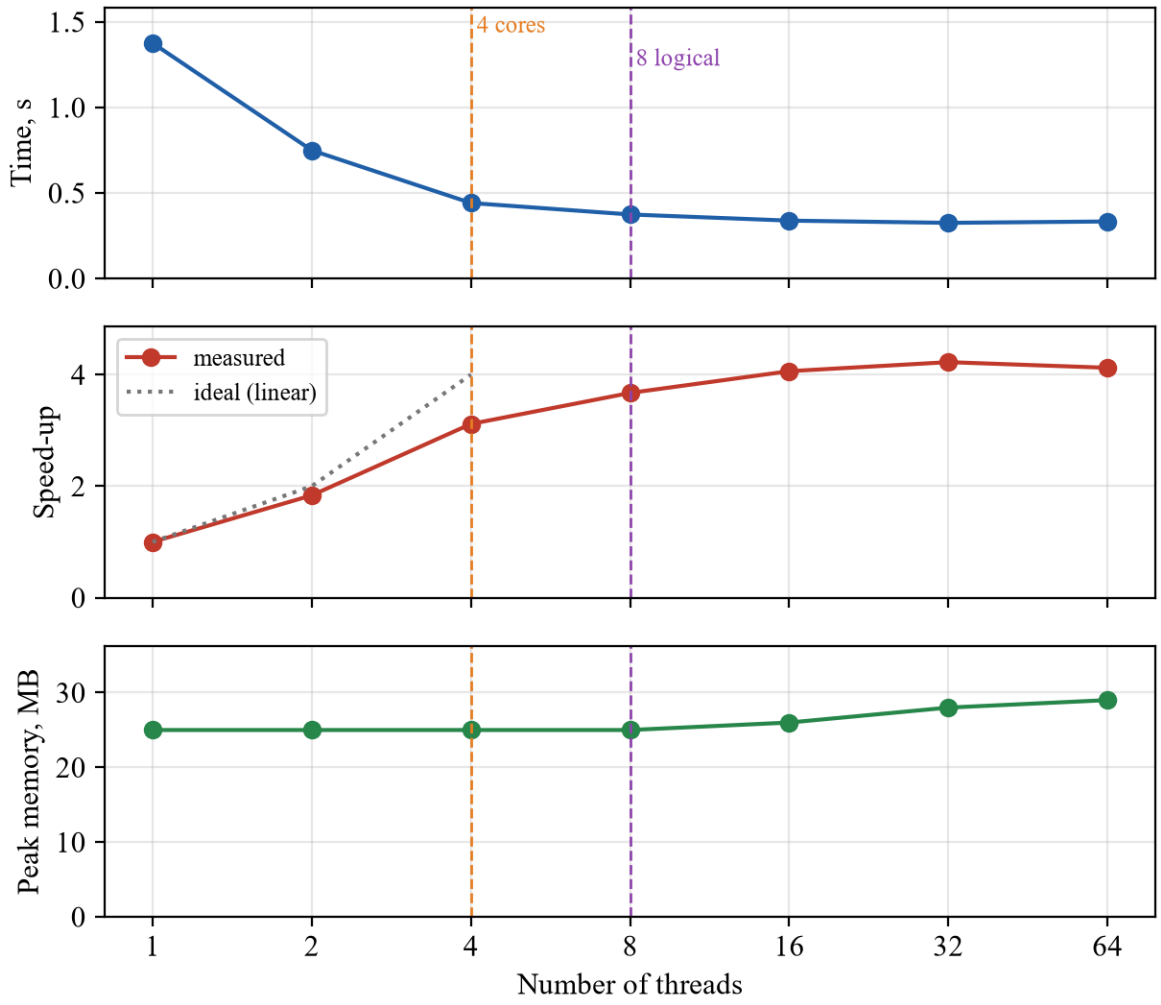


Figure 2.1 – Task 1a: time, speed-up and peak memory against the number of threads

2.3 Answers to the questions

1. At which number of threads does the speed-up stop growing?

Up to 4 threads the speed-up grows almost linearly: 1.84 with 2 threads and 3.12 with 4. The laptop has 4 physical cores, so up to 4 threads every thread gets a whole core. From 4 to 8 threads the

speed-up grows only from 3.12 to 3.67, by 18%: the new threads run on the second logical processor of cores that are already busy and share their execution units with the first thread. After 8, the number of logical processors, the speed-up practically stops: 4.06 with 16 threads, 4.22 with 32 and 4.12 with 64.

Even 4 threads give 3.12 and not 4. One busy core runs at a higher clock frequency (Turbo Boost, up to 4.4 GHz) than four busy cores, which share the power budget of the chip, and the laptop was on battery. The small gain after 8 threads comes from better balancing: with 8 equal parts, a thread that the operating system interrupts for a moment delays the whole run, while with 16 or 32 smaller parts the remaining parts can run on the processors that are already free.

2. Why do 32 and 64 threads not help? Are they slower?

Only 8 threads can run at the same time, one on each logical processor. The other threads wait for their turn while the operating system switches between them, so the total work is the same and is still done by 8 logical processors. Here 32 and 64 threads were not slower: 0.33 s, the same as with 16 threads within the noise of the measurement. Every extra thread does cost something: it has to be created, it opens the file, it has its own buffer and stack, and switching between threads takes time. But this is a few milliseconds against 330 ms of work. The cost shows in the spread of the times: with 64 threads the five runs took from 0.28 to 0.68 s.

3. Does the peak memory depend on the number of threads?

Almost not: 25 MB from 1 to 8 threads and 29 MB with 64 threads. Most of the 25 MB is the .NET runtime itself. The numbers are never loaded into memory: each thread reads its part in blocks of 64 KB and keeps only three counters, so the memory does not depend on the size of the file. Each thread adds only its 64 KB buffer, the small buffer of its `FileStream` and the part of its stack that it uses, about 65 KB per thread, which gives the 4 MB difference between 1 and 64 threads.

3 TASK 1B: THREE WAYS TO COUNT

3.1 Implementation

The numbers are first loaded into an `int[]` array with `File.ReadLines(path).Select(int.Parse).ToArray()`, which is not measured. The array is divided into 8 equal parts, as many as there are logical processors (`Environment.ProcessorCount`), and 8 threads count the signs of their parts in three ways:

1) `SharedNoLock`: all threads increment the same array of three counters, `c[Sign(a[j])]++`, without any synchronisation;

2) `SharedWithLock`: the same, but every increment is done inside `lock (sync)` [3];

3) `PerThread`: every thread counts into its own array `mine` and stores it at the end, and the 8 arrays are added together after all the threads have finished.

Version 1 is run three times in a row, as required, and then versions 2 and 3. The whole program was run five times.

3.2 Results

Table 3.1 gives the times and the totals of the first of the five runs of the program, and Figure 3.1 shows the median time of every version over all five.

Table 3.1 – Task 1b with 8 threads, first run of the program

Version	Time, s	Negative	Zero	Positive	Total	Correct
1, run 1	0.482	8 579 041	24	8 598 364	17 177 429	no
1, run 2	0.267	8 344 806	24	8 310 760	16 655 590	no
1, run 3	0.505	8 436 333	24	8 337 082	16 773 439	no
2, lock	5.039	25 005 087	24	24 994 889	50 000 000	yes
3, per thread	0.085	25 005 087	24	24 994 889	50 000 000	yes

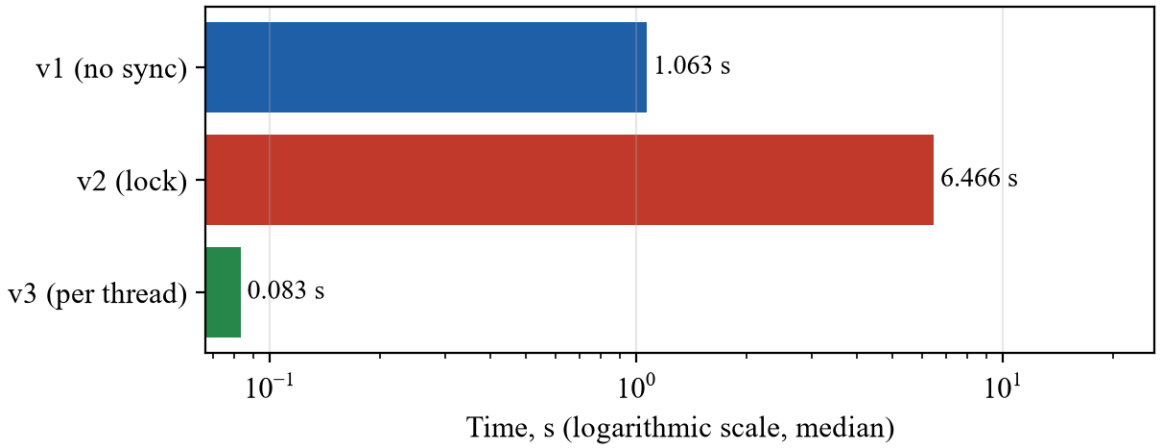


Figure 3.1 – Task 1b: the median time of every version over all the runs (logarithmic scale)

Over all five runs of the program, version 1 took from 0.27 to 1.10 s and counted from 15 533 151 to 20 487 268 numbers instead of 50 000 000, a different total every time. Version 2 took from 5.04 to 7.15 s and version 3 from 0.066 to 0.117 s, and both always gave the correct totals.

3.3 Answers to the questions

4. Rank the three versions by time. Which of them are correct?

From the fastest: version 3 with per-thread counters (median 0.083 s), version 1 without synchronisation (1.06 s, 13 times slower) and version 2 with a lock (6.47 s, 78 times slower than version 3). Only versions 2 and 3 are correct.

Version 1 is wrong because `c[k]++` is a read, an addition and a write, and when two threads do it at the same time one of the increments is lost (control question 3). Between 59% and 69% of the increments were lost, and the total was different in every run, because it depends on how the threads happened to interleave. The zero counter was always right, 24, because it is incremented only 24 times in the whole run, so two threads almost never touch it at the same moment: updates are lost only where there is contention. Version 1 is also slow although it has no lock. The three counters are in the same 64-byte cache line, and a core can write to a line only after taking it from the cache of the core that wrote it last, so the line keeps moving between the 8 logical processors.

Version 2 is correct, because the lock lets only one thread at a time do the read, the addition and the write. But the counting becomes sequential, and every one of the 50 000 000 increments pays for taking and releasing a contended lock, so this version is the slowest: 17 times slower than the whole of Task 1a with 8 threads, which also reads and parses the file. Version 3 is correct and the fastest. Every thread writes only to its own counters, which stay in the cache of its own core, the threads never wait for each other, and the results are added only once, after `Join()`, when all the threads have finished.

4 TASK 1C: PRODUCER AND CONSUMERS

4.1 Implementation

The queue is a `BlockingCollection<List<string>>` created with `boundedCapacity: 100` [2]. The reader thread reads numbers.txt with `StreamReader.ReadLine`, collects the lines into a list of 10 000 and adds the list to the queue; when the queue already holds 100 batches, `Add` blocks until a worker takes one. At the end of the file the reader adds the last batch and calls `CompleteAdding()`, which closes the queue. The workers take batches in a loop over `GetConsumingEnumerable()`, which blocks while the queue is empty and ends when the queue is closed and empty. Every worker parses the lines of its batches with `int.Parse` and counts them into its own counters, which are added together at the end, as in version 3 of Task 1b.

4.2 Results

Every configuration was run five times (Table 4.1, Figure 4.1). The number of threads is the number of workers; the reader is one more thread.

Table 4.1 – Task 1c, median of five runs

Workers	Time, s	Speed-up	Peak memory, MB	Times of the five runs, s	Totals correct
1	3.69	1.00	76	3.69, 3.55, 3.85, 3.32, 3.70	yes
2	3.81	0.97	40	3.81, 3.35, 4.24, 3.34, 3.86	yes
4	4.17	0.88	42	4.50, 4.17, 4.04, 3.98, 4.25	yes
8	5.38	0.69	52	6.20, 5.35, 5.38, 5.54, 4.92	yes
16	7.42	0.50	72	8.02, 7.51, 7.41, 7.42, 7.29	yes
32	9.20	0.40	105	9.20, 9.23, 8.78, 8.83, 9.39	yes
64	9.79	0.38	159	9.79, 9.82, 9.94, 9.28, 9.29	yes

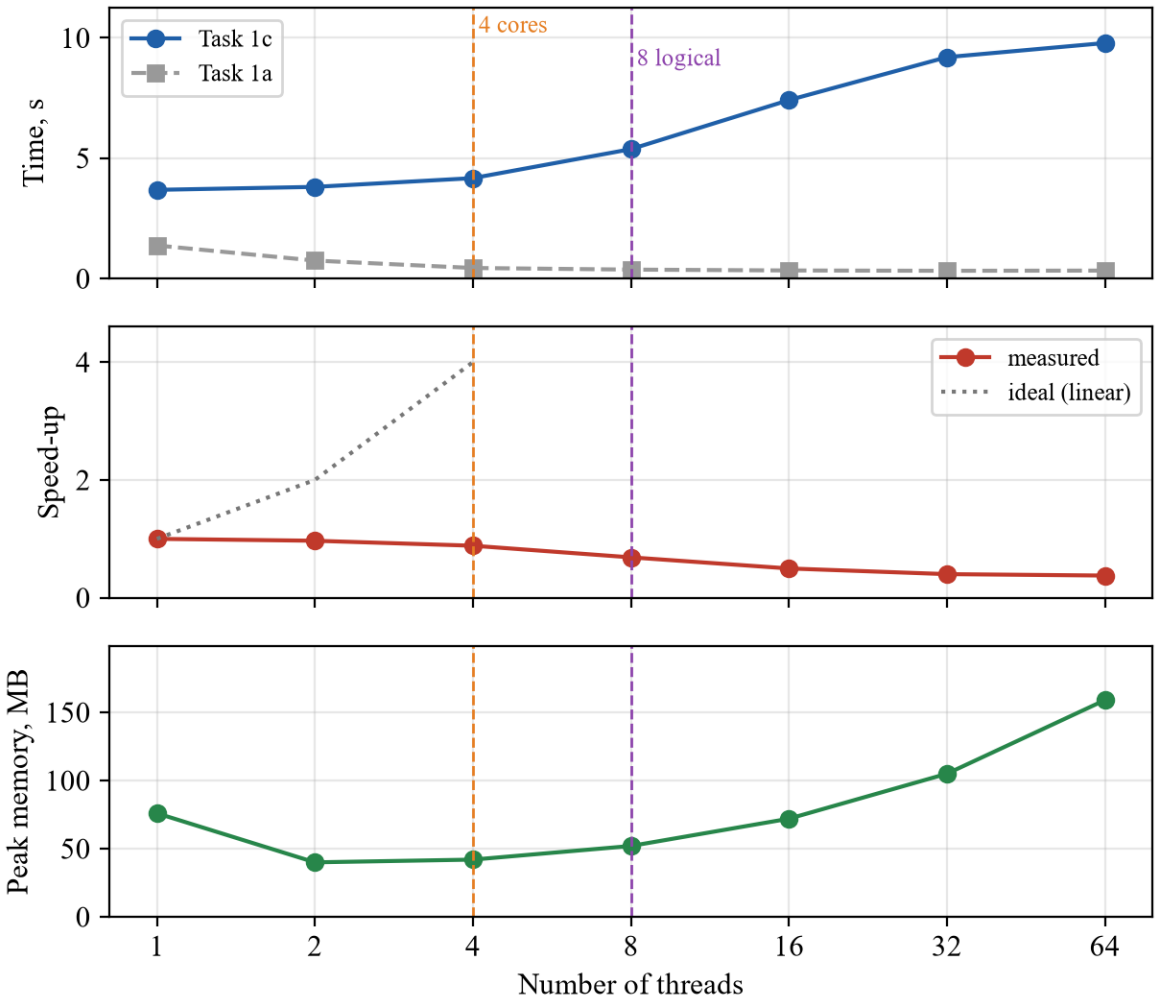


Figure 4.1 – Task 1c: time, speed-up and peak memory against the number of workers, with Task 1a for comparison

4.3 Answers to the questions

5. Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is much faster: 0.33 s at best against 3.69 s, eleven times less. Even Task 1a on one thread (1.38 s) is faster than Task 1c with any number of workers. In Task 1c all the reading is done by one thread, and it is the most expensive part of the work: `ReadLine` decodes the bytes into .NET strings and creates 50 000 000 string objects, which the garbage collector then has to free. The workers only parse short strings, which is much cheaper, so one worker keeps up with the reader, and the reader sets the time, about 3.5 s. In Task 1a there is no such sequential part: every thread reads and parses its own raw bytes, without creating any objects and without any synchronisation.

Task 1c does not get faster at all. 1 worker gave 3.69 s and 2 workers 3.81 s, the same within the spread of the runs, and from 4 workers on it gets slower, up to 9.79 s with 64 workers. More workers cannot speed up the reader, but they take processor time from it. A worker that finds the queue empty first spins for a short time before it goes to sleep, and with 8 or more workers on 4 cores these spinning threads compete with the reader for the processor. All the workers also compete for the lock of the queue, and the garbage collector, which runs often because the reader creates so many strings, has to stop every thread before each collection, which takes longer the more threads there are.

The peak memory shows where the bottleneck was. With 1 worker it was 76 MB, and with 2 workers 40 MB. One worker was slightly slower than the reader, so the queue filled up to its limit of 100 batches, a million strings of about 36 bytes each, which is the extra 36 MB. With 2 or more workers the queue stayed almost empty, because every batch was taken as soon as it was added. From 8 workers on, the memory grows again, up to 159 MB with 64 workers. Every worker holds the batch it is counting, and with more workers than logical processors a worker often holds its batch while it waits for its turn on a processor. Batches that live longer survive garbage collections and are moved to the older generations of the heap, which are collected rarely, so the heap grows.

6. What would happen with batches of a single line? And with a queue that has no size limit?

With batches of a single line, every number would need its own `Add` and `Take`, 50 000 000 of each instead of 5 000. Each of them takes the lock of the queue, updates its state and may have to wake a waiting thread, which costs much more than parsing one number. The reader and the workers would spend most of their time fighting for the queue, so the program would be many times slower, and more workers would make it even worse. Batches exist to pay the cost of synchronisation once per 10 000 numbers.

With a queue that has no size limit, the reader would never wait. When the reader is the bottleneck, as here with 2 or more workers, nothing would change, because the queue is almost always

empty anyway. But when the workers are slower than the reader, as with 1 worker, the queue would grow without limit: in the worst case the whole file would end up in memory as 50 000 000 strings, about 2 GB instead of the 36 MB that the limit of 100 batches allows. The limit is what keeps the memory of the program bounded; this is called back-pressure.

5 TASK 2: MANY FILES ON SLOW STORAGE

5.1 How the task was run

The 8 files were copied to the USB flash drive from Table 1, a 4 GB FAT32 drive that reports itself as "AI Mass Storage". Windows connected it to a High-Speed port (`HS02` in its device path), so the drive works as a USB 2.0 device although the port of the laptop supports USB 3. In this task the files were read only from the drive.

So that no data came from the file cache, the drive was removed from the system and added back before every run. This was done with the command `pnputil /restart-device` on the USB device of the drive, run as administrator. It stops the device, removes its volume together with everything cached for it, and starts the device again, which for the cache is the same as ejecting the drive and plugging it back in. The script `run_task2.ps1` from the source folder does the same with a real unplug: before every run it waits until the drive disappears and comes back.

A file is the unit of work. With n threads, thread i takes the files i, i + n, i + 2n and so on, so with 1, 2 and 4 threads every thread gets 8, 4 and 2 files. A thread reads a whole file with one call to `File.ReadAllBytes`, into one byte array, and passes the array to `SignCounter`.

5.2 Results

The results are given in Table 5.1 and Figure 5.1. The read speed is the size of the data, 352.3 MB, divided by the time.

Table 5.1 – Task 2 on the USB drive, one run per configuration

Threads	Time, s	Speed-up	Peak memory, MB	Read speed, MB/s
1	20.48	1.00	377	17.2
2	18.93	1.08	376	18.6
4	18.81	1.09	377	18.7

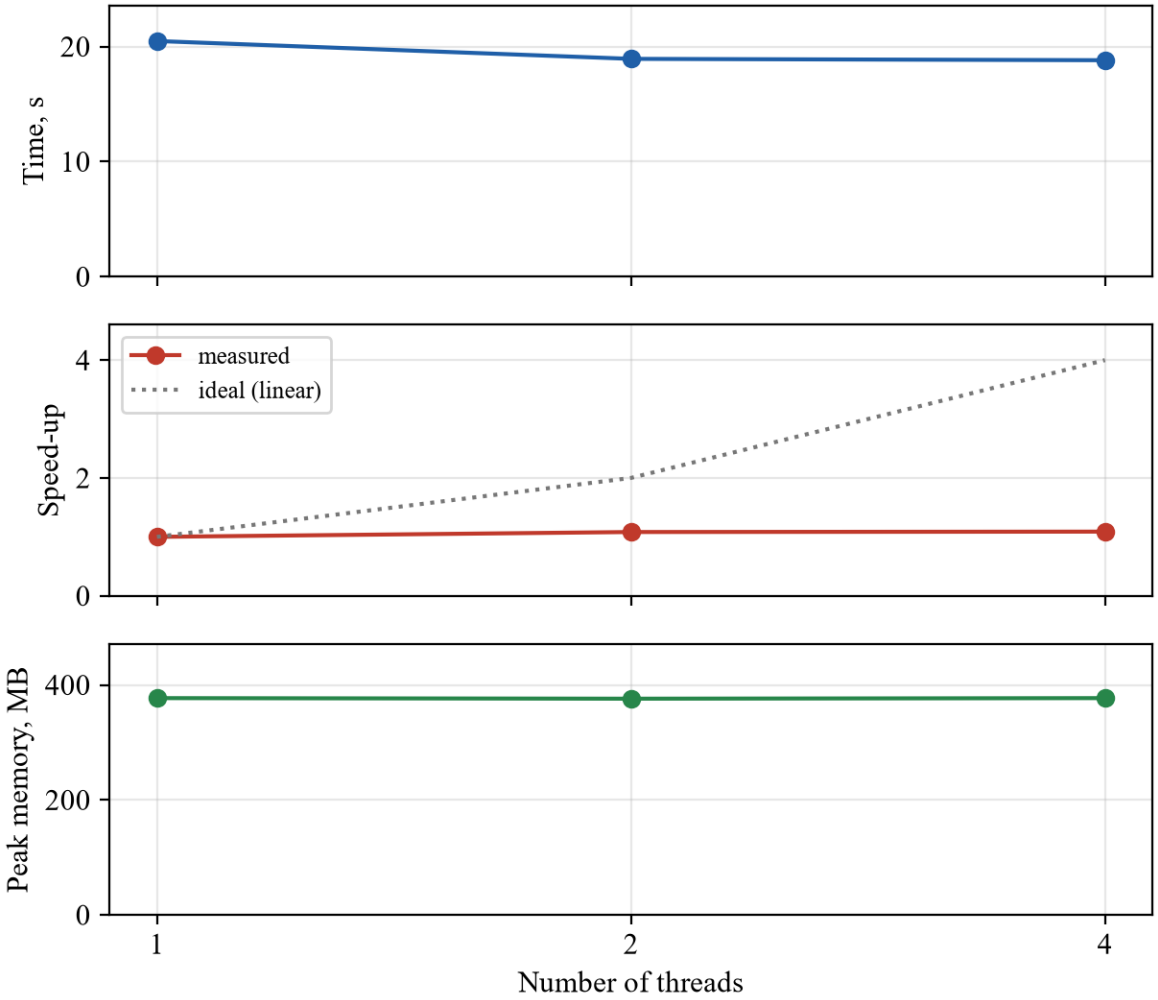


Figure 5.1 – Task 2 on the USB drive: time, speed-up and peak memory against the number of threads

For comparison, the command was run again right after the third run, without restarting the drive, so the same files came from the file cache. It took 1.64 s with 1 thread and 0.60 s with 4 threads, more than ten times less than from the drive.

5.3 Answers to the questions

1. Is the work on the USB drive CPU-bound or I/O-bound?

With one thread Task 2 took 20.48 s, while Task 1a with one thread took 1.38 s for the same 50 000 000 numbers from the laptop's own disk, in practice from its file cache, about 15 times less. The parsing is the same in both tasks: the same class `SignCounter` reads the same 352 MB. The

cached run of Task 2 shows how long this parsing takes on one thread: 1.64 s. So of the 20.48 s only about 1.6 s, less than 10%, is work of the processor, and the rest, about 19 s, the thread waits for the drive. The drive gives 17–19 MB/s, while one thread can parse about 215 MB/s. On the USB drive the work is therefore I/O-bound.

2. Do more threads help on the USB drive?

Almost not: 2 threads were 8% faster and 4 threads 9% faster than one. The small gain has a simple cause. With one thread the drive stands idle while a file is being parsed, about 0.2 s per file and 1.6 s for all eight. With two threads one file can be parsed while the next one is being read, and the time fell by 1.55 s, which is the parsing time. After that nothing is left to overlap, so 4 threads gave the same time as 2.

Lecture 1 showed that I/O-bound work benefits from concurrency when threads wait for different things, for example many network requests that each wait for their own distant server. Then the waits overlap. Here all the threads wait for the same device, and the limit is its bandwidth: one thread already keeps the USB 2.0 drive busy all the time, and more threads only share the same 18–19 MB/s among themselves. Reading several files at once can even be slower, because the drive has to jump between them.

The peak memory was about 377 MB with every number of threads. Each file is read into a new 44 MB array, and in .NET such large arrays are freed only by a full garbage collection. The machine had plenty of free memory, so the collector did not run one, and all eight arrays, $8 \cdot 44 = 352$ MB, stayed in memory until the end. In principle the task needs only one buffer per thread, from 44 MB with one thread to 176 MB with four, but the measured peak shows the behaviour of the garbage collector, not of the threads.

6 ANSWERS TO THE CONTROL QUESTIONS

1. What is the difference between a physical core and a logical processor? Where did it show in the charts?

A physical core is a complete processor with its own execution units and its own L1 and L2 caches. With Hyper-Threading every physical core keeps two sets of registers and runs two threads at once, so the operating system sees two logical processors per core. The two threads of one core share its execution units and caches, so the second thread only uses the parts of the core that the first one leaves idle, for example while it waits for memory. The i5-1145G7 has 4 physical cores and 8 logical processors. In Task 1a (Figure 2.1) the speed-up grew almost linearly up to 4 threads, one per physical core, and reached 3.12. From 4 to 8 threads, when the second logical processor of every core came into use, it grew only to 3.67, by 18%, and after 8 it stopped. The second logical processor of a core is therefore worth much less than a second core.

2. How was it ensured that no line was counted twice or skipped at the boundary between two parts?

The rule is that a line belongs to the part in which its first byte lies. The ends of every part are computed by the same function, `LineStart`, which moves a cut forward to the beginning of the next line: it looks at the byte just before the cut and, if it is not a newline, skips to the byte after the next newline. The end of part i and the start of part i + 1 are computed from the same cut by the same function, so they are always equal: there is no gap and no overlap. A line that crosses a cut is read whole by the part in which it starts and skipped by the next part. The first part starts at 0 and the last one ends at the end of the file. The check is the totals: every run, with any number of threads, including 3 threads in a test, gave exactly 50 000 000 numbers and the same three counts.

3. How do unsynchronised shared counters lose updates?

The statement `c[k]++` looks like one operation, but the processor does it in three steps: it reads the counter from memory into a register, adds one to the register and writes the register back. Suppose the counter is 100 and two threads on two cores want to increment it:

a) thread A reads 100;

b) thread B reads 100, because A has not written yet;

c) thread A adds one and writes 101;

d) thread B adds one and writes 101.

Two increments were made, but the counter grew by one: the update of A was overwritten by B. With 8 threads and only three counters this happens all the time. In Task 1b the version without synchronisation lost more than half of the increments, and every run gave a different wrong total, because the result depends on how the threads happen to interleave.

4. Why are per-thread counters faster than shared counters protected by a lock?

With a lock, every one of the 50 000 000 increments has to take and release the lock. Taking it is an atomic instruction that forces the cores to agree on the value of the lock in memory, and when another thread holds it, the thread spins or goes to sleep and has to be woken up. The lock and the counters live in a few cache lines that every core writes, so these lines move from core to core on almost every increment, and in practice only one thread at a time makes progress. Per-thread counters are written only by their own thread, they stay in the L1 cache of its core, no thread ever waits for another, and the only shared step is adding $8 \cdot 3$ numbers at the end. In Task 1b the lock made the counting 78 times slower than per-thread counters.

5. What does "I/O-bound" mean for a USB drive and for a server waiting on the network? Why does concurrency help in only one of the two cases?

In both cases the program spends most of its time waiting for input and output instead of computing. The difference is what it waits for. A USB drive is one device with a fixed bandwidth, and one thread that reads one file after another already keeps it busy all the time. More threads do not make the device faster; they only divide the same 18–19 MB/s among themselves, as Task 2 showed. A server waiting on the network usually waits for latency: a request goes to another machine and the answer comes back after, for example, 100 ms, while the network and the other machine are far from busy. Many such waits can overlap, so 100 concurrent requests of 100 ms take about 100 ms in total, not 10 s. Concurrency helps when the waits can overlap and does not help when every thread waits for the same saturated resource.

6. What is the difference between dividing the work into equal parts in advance and taking it from a queue? When is each one better?

With equal parts in advance, as in Task 1a, every thread knows its work from the start and the threads never talk to each other, so there is no synchronisation cost. It is best when the work is known in advance, can be cut into pieces of equal cost, and every thread runs at the same speed. Its weakness is that the slowest part decides the total time: if one thread is slowed down, for example because the operating system gives its core to another program, all the others wait for it.

With a queue, as in Task 1c, threads take the next piece of work when they become free, so the load is balanced automatically, and the work does not have to be known in advance: it can arrive as a stream, from a file read line by line or from the network. The price is synchronisation on every take and, as Task 1c showed, the producer itself may become the bottleneck. A queue is better when the pieces have different costs, when the threads run at different speeds, or when the input is a stream; equal parts are better for a large, uniform job that is available at once.

CONCLUSIONS

On this laptop the counting was CPU-bound whenever the data came from memory or from the file cache. In Task 1a the time fell almost in proportion to the number of threads up to the 4 physical cores, fell a little more up to the 8 logical processors and then stopped, with a best speed-up of about 4. This is how CPU-bound work behaves: it is limited by the number of cores, and threads beyond the number of logical processors only wait for their turn.

Task 1b showed the price of sharing data between threads. Unsynchronised shared counters lost 59–69% of the increments. A lock made the result correct, but the counting 78 times slower than with per-thread counters, which were both correct and the fastest. The best synchronisation is to avoid sharing: each thread works on its own data, and the results are combined once at the end.

Task 1c showed that a producer–consumer pipeline is as fast as its slowest stage. The single reader, which turned every line into a string, was slower than all the parsing together, so more workers had nothing to gain, and they only made the program slower and larger. Task 1a, where every thread reads and parses its own part, was eleven times faster.

The work was I/O-bound only on the USB 2.0 drive in Task 2. There the same parsing took 20.5 s instead of 1.4 s, the drive delivered only 17–19 MB/s, and more threads gave at most 9%, which is exactly the parsing time that could be overlapped with reading. The two cases are told apart by the measurements: CPU-bound work gets faster with more cores, as Task 1a did, while I/O-bound work does not get faster with more threads, but becomes more than ten times faster as soon as the same data comes from memory instead of the device, as Task 2 did (20.5 s from the drive, 1.64 s from the cache).

REFERENCES

1. Laboratory Work 1: Concurrency in Practice. Network Programming course, Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.

2. Microsoft. BlockingCollection<T> Class [online]. Available: https://learn.microsoft.com/dotnet/api/system.collections.concurrent.blockingcollection-1

3. Microsoft. The lock statement [online]. Available: https://learn.microsoft.com/dotnet/csharp/language-reference/statements/lock

4. Microsoft. Process.PeakWorkingSet64 Property [online]. Available: https://learn.microsoft.com/dotnet/api/system.diagnostics.process.peakworkingset64

5. Microsoft. PnPUtil command syntax [online]. Available: https://learn.microsoft.com/windows-hardware/drivers/devtest/pnputil-command-syntax

APPENDIX A. SOURCE CODE

The file Program.cs, the whole program.

```
using System.Collections.Concurrent;
using System.Diagnostics;

const int Total = 50_000_000;

switch (args.Length > 0 ? args[0] : "")
{
    case "gen": Generate(args[1]); break;
    case "split": Split(args[1], args[2]); break;
    case "1a": { int t = int.Parse(args[2]); Run("1a", t, () => Task1a(args[1], t));
break; }
    case "1b": Task1b(args[1]); break;
    case "1c": { int t = int.Parse(args[2]); Run("1c", t, () => Task1c(args[1], t));
break; }
    case "2": { int t = int.Parse(args[2]); Run("2", t, () => Task2(args[1], t));
break; }
    default: Console.WriteLine("usage: Lab1 gen|split|1a|1b|1c|2 ..."); break;
}

static void Generate(string path)
{
    var random = new Random(42);
    using var w = File.CreateText(path);
    w.NewLine = "\n";
    for (int i = 0; i < Total; i++) w.WriteLine(random.Next(-1_000_000, 1_000_001));
}

static void Split(string path, string dir)
{
    Directory.CreateDirectory(dir);
    using var r = new StreamReader(path);
    for (int f = 1; f <= 8; f++)
    {
        using var w = File.CreateText(Path.Combine(dir, $"numbers_{f}.txt"));
        w.NewLine = "\n";
        for (int i = 0; i < Total / 8; i++) w.WriteLine(r.ReadLine());
    }
}

static long[] Task1a(string path, int threads)
{
    long size = new FileInfo(path).Length;
    var results = new long[threads][];
    RunThreads(threads, i =>
    {
        long from = LineStart(path, size * i / threads);
        long to = LineStart(path, size * (i + 1) / threads);
        results[i] = CountPart(path, from, to);
    });
    return Sum(results);
}

static long LineStart(string path, long pos)
{
    if (pos == 0) return 0;
    using var fs = File.OpenRead(path);
```

```
    if (pos >= fs.Length) return fs.Length;
    fs.Position = pos - 1;
    int b;
    while ((b = fs.ReadByte()) != '\n' && b != -1) { }
    return fs.Position;
}

static long[] CountPart(string path, long from, long to)
{
    var counter = new SignCounter();
    using var fs = File.OpenRead(path);
    fs.Position = from;
    var buffer = new byte[64 * 1024];
    long left = to - from;
    while (left > 0)
    {
        int n = fs.Read(buffer, 0, (int)Math.Min(buffer.Length, left));
        if (n == 0) break;
        counter.Feed(buffer.AsSpan(0, n));
        left -= n;
    }
    counter.End();
    return counter.Counts;
}

static void Task1b(string path)
{
    int[] numbers = File.ReadLines(path).Select(int.Parse).ToArray();
    int t = Environment.ProcessorCount;
    for (int run = 1; run <= 3; run++) Run($"1b-v1-run{run}", t, () =>
SharedNoLock(numbers, t));
    Run("1b-v2", t, () => SharedWithLock(numbers, t));
    Run("1b-v3", t, () => PerThread(numbers, t));
}

static long[] SharedNoLock(int[] a, int threads)
{
    var c = new long[3];
    RunThreads(threads, i =>
    {
        int from = Start(a.Length, i, threads), to = Start(a.Length, i + 1, threads);
        for (int j = from; j < to; j++)
            c[Sign(a[j])]++;
    });
    return c;
}

static long[] SharedWithLock(int[] a, int threads)
{
    var c = new long[3];
    var sync = new object();
    RunThreads(threads, i =>
    {
        int from = Start(a.Length, i, threads), to = Start(a.Length, i + 1, threads);
        for (int j = from; j < to; j++)
            lock (sync) c[Sign(a[j])]++;
    });
    return c;
}

static long[] PerThread(int[] a, int threads)
```

```
{
    var results = new long[threads][];
    RunThreads(threads, i =>
    {
        var mine = new long[3];
        int from = Start(a.Length, i, threads), to = Start(a.Length, i + 1, threads);
        for (int j = from; j < to; j++)
            mine[Sign(a[j])]++;
        results[i] = mine;
    });
    return Sum(results);
}

static long[] Task1c(string path, int workers)
{
    var queue = new BlockingCollection<List<string>>(boundedCapacity: 100);
    var reader = new Thread(() =>
    {
        using var r = new StreamReader(path);
        var batch = new List<string>(10_000);
        string? line;
        while ((line = r.ReadLine()) != null)
        {
            batch.Add(line);
            if (batch.Count == 10_000) { queue.Add(batch); batch = new
List<string>(10_000); }
        }
        if (batch.Count > 0) queue.Add(batch);
        queue.CompleteAdding();
    });
    reader.Start();

    var results = new long[workers][];
    RunThreads(workers, i =>
    {
        var mine = new long[3];
        foreach (var batch in queue.GetConsumingEnumerable())
            foreach (string s in batch) mine[Sign(int.Parse(s))]++;
        results[i] = mine;
    });
    reader.Join();
    return Sum(results);
}

static long[] Task2(string dir, int threads)
{
    string[] files = Directory.GetFiles(dir, "numbers_*.txt").Order().ToArray();
    var results = new long[threads][];
    RunThreads(threads, i =>
    {
        var counter = new SignCounter();
        for (int f = i; f < files.Length; f += threads)
        {
            byte[] data = File.ReadAllBytes(files[f]);
            counter.Feed(data);
            counter.End();
        }
        results[i] = counter.Counts;
    });
    return Sum(results);
}
```

```
static void RunThreads(int count, Action<int> body)
{
    var threads = new Thread[count];
    for (int i = 0; i < count; i++)
    {
        int id = i;
        threads[i] = new Thread(() => body(id));
        threads[i].Start();
    }
    foreach (var t in threads) t.Join();
}

static void Run(string name, int threads, Func<long[]> work)
{
    var sw = Stopwatch.StartNew();
    long[] c = work();
    sw.Stop();
    long peakMb = Process.GetCurrentProcess().PeakWorkingSet64 / (1024 * 1024);
    long total = c[0] + c[1] + c[2];
    Console.WriteLine($"{name} threads={threads} time_ms={sw.ElapsedMilliseconds} " +
                      $"neg={c[0]} zero={c[1]} pos={c[2]} total={total}
peak_mb={peakMb}" +
                      (total == Total ? "" : " WRONG"));
}

static int Start(int length, int part, int parts) => (int)((long)length * part /
parts);

static int Sign(int x) => x < 0 ? 0 : x == 0 ? 1 : 2;

static long[] Sum(long[][] parts)
{
    var c = new long[3];
    foreach (var p in parts)
        for (int k = 0; k < 3; k++) c[k] += p[k];
    return c;
}

class SignCounter
{
    public long[] Counts = new long[3];
    int value;
    bool minus, digits;

    public void Feed(ReadOnlySpan<byte> data)
    {
        foreach (byte b in data)
        {
            if (b == '\n') End();
            else if (b == '-') minus = true;
            else if (b >= '0' && b <= '9') { value = value * 10 + (b - '0'); digits =
true; }
        }
    }

    public void End()
    {
        if (!digits) return;
        int x = minus ? -value : value;
        Counts[x < 0 ? 0 : x == 0 ? 1 : 2]++;
```

```
            value = 0; minus = false; digits = false;
        }
    }
```